COMP/ELEC/MECH 450/550

Project 3: Barking up a Random Tree

Darshon Singh (ds161) and Winson Lin (wl85)

Problem, arm model and RTP

We implemented the Random Tree Planner (RTP) in C++ and planned Panda-arm motions around 3D obstacles. We then compared RTP with OMPL RRT, RRT-Connect and PRM on repeated clutter-scene queries. The seven revolute joint angles define $\mathcal{C} = \prod_{i=1}^{7} [q_i^{\min}, q_i^{\max}] \subset \mathbb{R}^7$. The base and gripper are fixed. URDF limits, in radians, are:

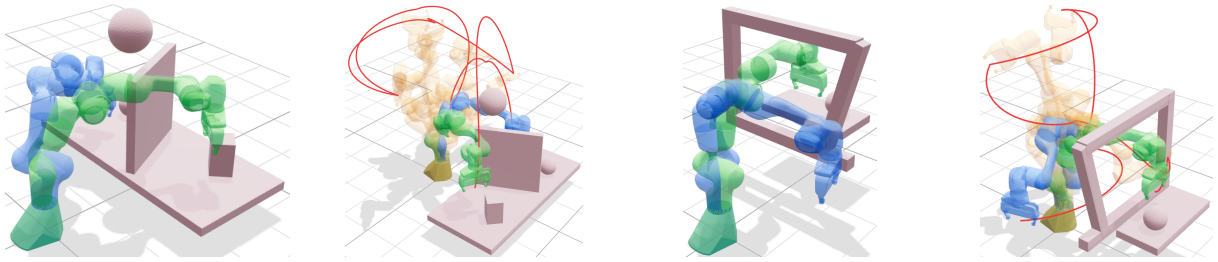
joint	1	2	3	4	5	6	7
min	−2.9671	−1.8326	−2.9671	−3.1416	−2.9671	−.0873	−2.9671
max	2.9671	1.8326	2.9671	.0873	2.9671	3.8223	2.9671

OMPL uses Euclidean distance and linear interpolation in this bounded joint box. Forward kinematics places 59 robot spheres for collision checks against environment spheres and oriented boxes. Self-collision and dynamic limits are outside the supplied model.

RTP stores states and parent pointers in a vector. Each iteration selects a vertex uniformly, samples a target with goal bias, checks the full motion, and adds only an accepted endpoint. Parent pointers recover an exact goal path or, on timeout, the path to the closest vertex. There is no nearest-neighbor selection or range limit. For Part 2, we rebuilt RTP without changing the algorithm. Tests covered 25 cycles of clearing and reusing a planner, exact and approximate paths, continued solving, starts already at the goal, invalid starts, tree export and state ownership. All allocated states were released when the test objects were destroyed. `clear()` resets the sampler, frees all states/nodes and clears the goal pointer.

Custom scenes and saved RTP paths

Wall: a table, a wall rotated 25° about z, a hanging ball and additional clutter. Both queries sweep left to right; hand-sphere centroids go from $(.46, .37, .27)$ to $(.62, -.29, .39)$ m (q1), and $(.54, .42, .39)$ to $(.51, -.23, .52)$ m (q2). **Window:** a frame at $x = .45$ m, an approximately $.6 \times .5$ m opening, a tilted post, and a shelf/ball behind it. The arm starts to either side and reaches through the frame to $(.56, .08, .44)$ and $(.62, .08, .50)$ m. All endpoints are valid, but each direct joint-space edge collides. The YAML files list the joint angles for each start and goal.



(a) Wall q1 start/goal (b) Wall q1 RTP path (c) Window q2 start/goal (d) Window q2 RTP path

Figure 1: RTP in our custom scenes: blue start, green goal, orange raw RTP waypoints, red gripper path. The submission includes full-size images and paths for both queries in each scene.

For wall q1/q2 and window q1/q2, the four saved exact paths (bias .05, 10 s limit) took .676, .201, .117 and .603 s, with lengths of 30.77, 9.61, 11.80 and 24.46 rad. Although they have only 4–8 waypoints, the arm makes large swings and loops. Window q2 seed 2 instead returned an approximate one-state path after 10.065 s. These individual examples do not establish a success rate. State ownership and selecting the approximate endpoint were the main implementation challenges.

Experiment design and quantitative comparison

We used the supplied `benchmark.py` and `configs/planners.json` to test all five clutter queries: 20 settings $\times$ 5 queries $\times$ 5 trials = **500 runs**. A separate 1 s pilot (100 runs, seed 18001) gave 86 exact and 14 approximate outcomes; we then used a 5 s limit for every planner to give the harder settings more time. The comparisons below use only the 500 runs at this limit.

We ran the experiments in the course's `ompl_env` environment on macOS ARM64, using Python 3.12.12, OMPL 2.0.2 and AppleClang 17 with a Release build. Trials ran one at a time, with no builds, tests or rendering running alongside them. Normal background OS activity was not controlled. We initialized OMPL once with seed 18002; its random generators then advanced between trials. The recorded seed therefore does not identify an independently seeded trial. Raw OMPL logs, source hashes and software versions are included with the results.

Goal bias b is the probability of sampling the goal. For RTP, we tested .01–.5 to vary how often it attempts to reach the goal. RRT varies range $r = .2, .4, 1, 2.5$ rad at $b = .05$, and bias $.01, .05, .2$ at $r = 1$, separating the two effects. RRT-Connect uses $r = .2, .4, 1, 2.5, 5$ rad, from fine steps to 37% of the joint-box diagonal. PRM uses $k = 5, 10, 20, 40$ neighbors to vary local connectivity. All settings are listed in the planner configuration file.

The collision model, bounds, resolution .005 and edge-clearance sampler stayed fixed. The 13.4165 rad box diagonal gives a nominal maximum checking interval of .0671 rad. We measured paths without simplification or smoothing. Time is OMPL's measured `solve()` duration, including collision checking. Length is the sum of joint-space edge lengths (rad). Smoothness uses OMPL's `PathGeometric::smoothness()`. Minimum clearance is the smallest sampled distance from a robot sphere's surface to an obstacle along the path (m), shown below in cm. We did not substitute OMPL's mean waypoint clearance for this minimum.

			all-run time (s)			exact-path medians		
setting	exact (%) of 25	A/F	median	mean	p90	rad	smooth.	cm
RTP_gb0.01	25/25 (100%)	0/0	0.878	1.150	2.639	23.46	0.80	4.83
RTP_gb0.05	25/25 (100%)	0/0	0.304	0.322	0.634	13.35	0.59	10.57
RTP_gb0.1	25/25 (100%)	0/0	0.132	0.214	0.527	12.66	0.57	8.22
RTP_gb0.2	25/25 (100%)	0/0	0.080	0.103	0.195	13.58	0.66	5.91
RTP_gb0.5	25/25 (100%)	0/0	0.040	0.057	0.102	9.39	0.48	6.40
RRT_gb0.05_r0.2	17/25 (68%)	8/0	0.102	1.712	5.034	4.29	651.55	1.18
RRT_gb0.05_r0.4	20/25 (80%)	5/0	0.144	1.279	5.021	4.86	100.35	2.06
RRT_gb0.05_r1.0	24/25 (96%)	1/0	0.099	0.446	0.340	4.12	7.56	3.93
RRT_gb0.05_r2.5	25/25 (100%)	0/0	0.439	0.561	1.043	8.06	2.03	6.67
RRT_gb0.01_r1.0	23/25 (92%)	2/0	0.484	0.956	2.439	5.61	9.27	1.97
RRT_gb0.2_r1.0	19/25 (76%)	6/0	0.118	1.546	5.034	5.35	8.20	3.84
RRTConnect_r0.2	25/25 (100%)	0/0	0.022	0.040	0.086	4.11	557.87	0.37
RRTConnect_r0.4	25/25 (100%)	0/0	0.024	0.030	0.049	4.60	110.23	1.33
RRTConnect_r1.0	25/25 (100%)	0/0	0.022	0.025	0.047	4.60	11.24	2.91
RRTConnect_r2.5	25/25 (100%)	0/0	0.019	0.020	0.031	5.61	1.98	8.83
RRTConnect_r5.0	25/25 (100%)	0/0	0.025	0.030	0.049	8.83	0.58	6.09
PRM_k5	25/25 (100%)	0/0	0.070	0.087	0.162	8.45	0.45	9.05
PRM_k10	25/25 (100%)	0/0	0.058	0.073	0.125	8.81	0.40	6.17
PRM_k20	25/25 (100%)	0/0	0.075	0.100	0.214	8.60	0.49	8.89
PRM_k40	25/25 (100%)	0/0	0.059	0.068	0.116	8.80	0.42	6.46

Table 1: Results for the clutter scene with a 5 s limit. A/F: approximate/failure counts. Path-quality medians use the exact solutions counted in the second column. A failure has no exact or approximate path. Time includes every trial at its actual elapsed duration.

We used the database status labels and checked them against `solved` and `approximate_solution`: exact is code 6, approximate is 5. Approximate paths stop short of the goal, so we counted them as unsuccessful and excluded them from path-quality summaries. They remain in the CSV files. Timing summaries include every run at its actual duration, even when it exceeds 5 s: a motion check or termination check can finish after the time limit. A setting can look good on path quality if it succeeds only on easier queries.

Parameter effects, variability and trade-offs

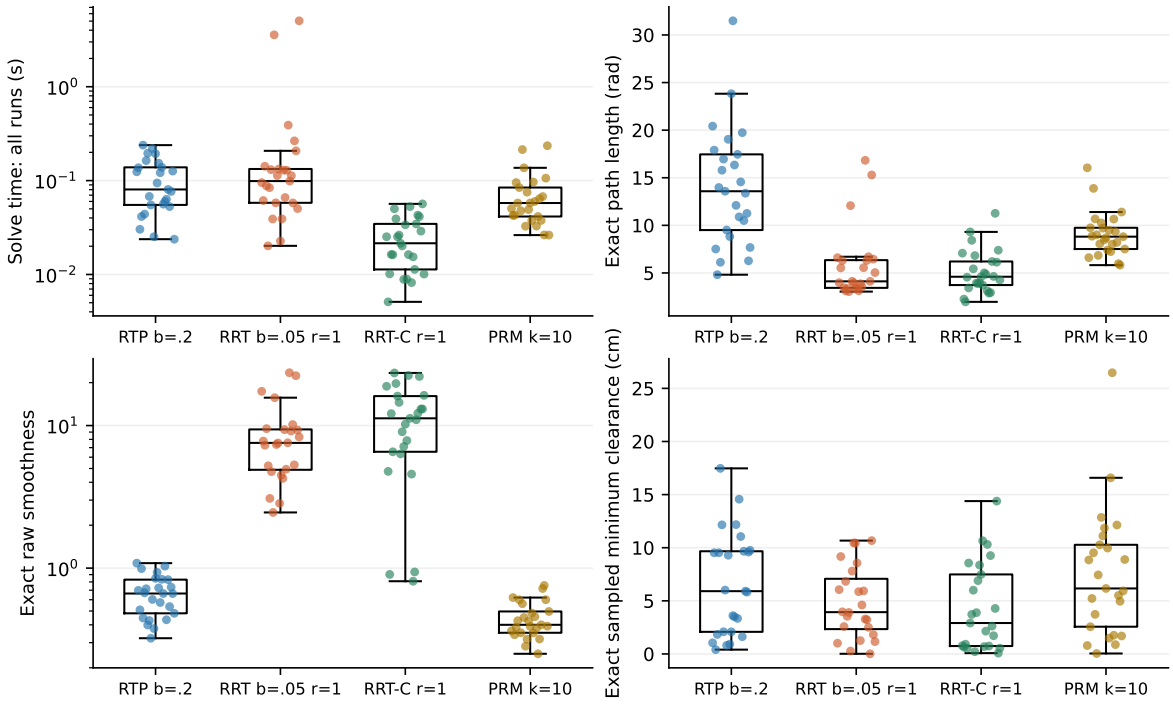


Figure 2: Four selected settings: RTP $b = .2$, RRT $b = .05, r = 1$, RRT-Connect $r = 1$, PRM $k = 10$. Points are individual runs; boxes show quartiles and medians. Solve time uses all 25 runs; quality uses exact runs only (counts in Table 1). Time and smoothness use log scales.

RTP. Raising bias from .01 to .5 reduced median solve time from .878 to .040 s and median length from 23.46 to 9.39 rad, with 25/25 exact solutions at every setting. Sampling the goal more often helped in this scene. The bias-.5 setting also gave RTP's best median smoothness (.478); clearance instead favored .05 (10.57 cm). Increasing bias did not improve every metric. RTP's full connections from random vertices often produce long detours.

RRT. With bias fixed at .05, increasing range from .2 to 2.5 rad increased the number of exact solutions from 17 to 20 to 24 to 25, but the 2.5 rad setting had a longer median path (8.06 versus 4.12 rad at range 1). At fixed range 1, biases .01/.05/.2 gave 23/24/19 exact trials. Larger bias did not consistently help. Bias .2 had median time .118 s but mean 1.546 s; .05 had .099 s median and .446 s mean. Approximate trials explain much of the tail. Nearest-vertex extension can repeatedly approach obstacles from similar directions, but we did not test this explanation directly.

RRT-Connect. Every setting found exact solutions in all 25 trials. Range 2.5 had the lowest median solve time (.0189 s), though the timing distributions overlap. Bidirectional growth attempts to connect trees from both endpoints, which can reduce exploration needed for these queries. Range .2 gave its shortest median path (4.114 rad); range 5 increased this to 8.827 rad. Median clearance rose from .37 cm at range .2 to 8.83 cm at 2.5, then fell to 6.09 cm at 5. Larger ranges did not always give greater clearance.

PRM. Every neighbor count gave 25 exact solutions out of 25 trials. Median lengths varied only from 8.45 to 8.81 rad; no monotonic improvement followed increasing k. The best observed medians were $k = 10$ for time (.058 s) and smoothness (.400), and $k = 5$ for length and clearance (9.05 cm). PRM starts with a new roadmap in each trial and stops after finding a path.

Metric interpretation. Smoothness sums $[2\theta/(a+b)]^2$ over interior waypoints. Many short edges can score worse than a few long ones: RRT-Connect fell from 557.87 at range .2 to .58 at 5. This measures waypoint geometry, not acceleration. All 478 exact paths passed the prescribed check, but 14 failed OMPL's post-run check with four times the segment count: 4 RRT and 10 RRT-Connect, including 6 at $r = .2$. We kept these paths in the exact-solution summaries because OMPL reported them as exact at the required resolution. That status does not guarantee collision freedom. Sampled minimum clearance can miss collisions. Path-quality summaries use only exact solutions, which may come from different subsets of queries.

Success by query and time to an exact solution

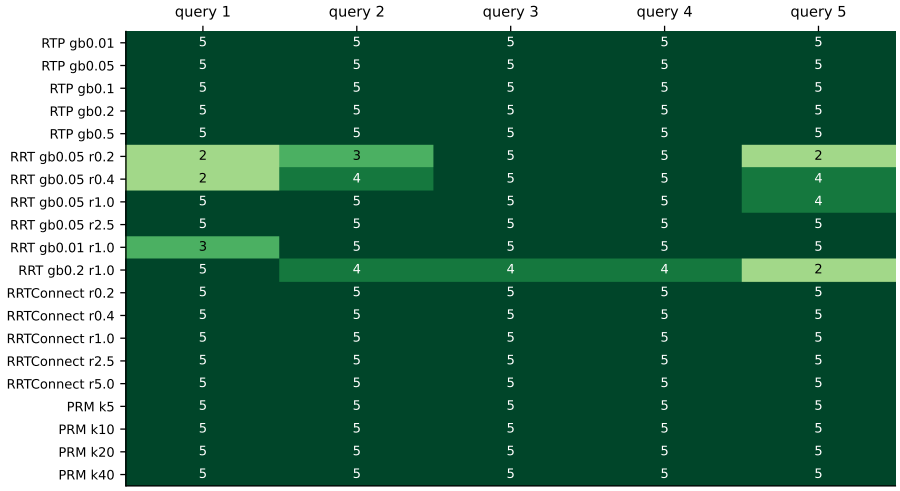


Figure 3: Exact solutions out of five trials in each query/setting; each cell contributes equally to its setting's 25-run summary. The complete per-query metric summaries are in `results/part2/per_query.csv`.

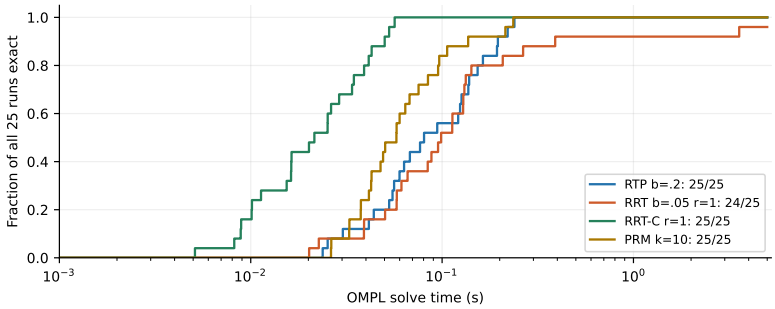
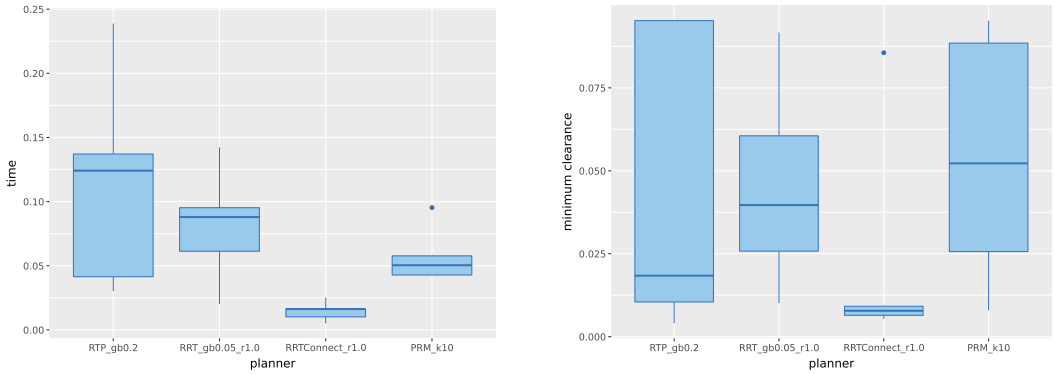


Figure 4: Fraction of all 25 trials that reached the exact goal by the indicated measured solve time. Unsuccessful trials stay in the denominator. This differs from a successful-run-only timing plot, which would hide unreliability.



(a) solve time (s)

(b) minimum clearance (m)

Figure 5: Planner Arena query1 plots from our uploaded database, five runs per setting (all five solved).

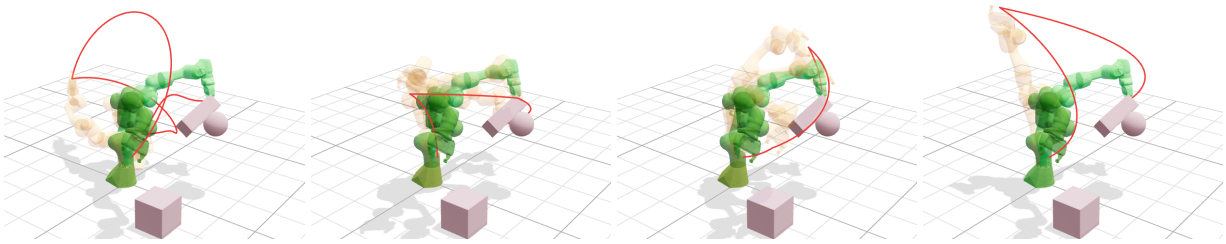
Some queries were harder for RRT than others. At bias .05 and range .2, it solved 2/5, 3/5 and 2/5 trials on q1/q2/q5, but all five on q3 and q4. Bias .2 at range 1 solved only 2/5 on q5. With bias .05 and range 1, only one q5 trial was approximate; range 2.5 solved all 25 trials. The overall success rate hides these differences. The approximate run at bias .05 and range 1 took 5.03806 s, which is included in the timing summaries.

RTP, RRT-Connect and PRM solved all five trials on every query at every setting. Five trials per query are too few to

guarantee that these planners will always succeed. All settings used the same queries, but individual trials did not use matching seeds. These results show differences in our tests; they do not establish statistical significance or a ranking that applies to every scene.

Sources. Assignment and supplied OMPL workflow; OMPL PathGeometric metrics. Planner Arena: Moll, Şucan and Kavraki, "Benchmarking Motion Planning Algorithms," IEEE RAM 22(3), 96–102, 2015.

Representative paths and conclusions



(a) RTP $b = .2$ (b) RRT $b = .05, r = 1$ (c) RRT-Connect $r = 1$ (d) PRM $k = 10$

Figure 6: Example paths for clutter query1 in Viser: blue start, green goal, orange interior waypoints and red gripper trace. Same camera and scale. The selected raw path was also inspected with `plan_arm.py --visualize`. Full-size overviews and playback captures are included.

For each setting, we ran query1 with seeds 18101–18105 and chose the exact path closest to its median benchmark length for that query. These example runs are saved separately and excluded from the benchmark statistics. RTP/RRT/RRT-Connect/PRM found 5/3/5/5 exact paths. Selected lengths are 13.78/3.72/3.63/11.55 rad; benchmark medians are 13.98/5.53/3.42/9.74 rad. Selecting by length does not make these paths typical of every metric.

RTP shows a broad gripper loop from its unrestricted connections. RRT and RRT-Connect take shorter routes here; PRM uses longer roadmap connections. Viser playback interpolates the saved raw joint paths for display.

Best observed settings. Across all settings, the fastest median was RRT-Connect $r = 2.5$ (.0189 s), the shortest was RRT-Connect $r = .2$ (4.114 rad), smoothest was PRM $k = 10$ (.400), and greatest sampled minimum clearance was RTP $b = .05$ (10.57 cm). Each used 25 exact paths. Excluding paths that failed the finer check changes the shortest median to RRT $b = .05, r = 1$ (4.117 rad, 23 paths); the other overall winners stay the same. The best settings for each planner in the main comparison were:

planner	time	length	smoothness	clearance
RTP (b)	.5	.5	.5	.05
RRT (b, r)	(.05,1)	(.05,1)	(.05,2.5)	(.05,2.5)
RRT-Connect (r)	2.5	.2	5	2.5
PRM (k)	10	5	10	5

RRT-Connect $r = 2.5$ gave a good balance in our tests: 25/25 exact, .019 s median, 5.61 rad length and 8.83 cm clearance. Range .2 gave shorter paths but 6/25 failed the finer check; all 25 at range 2.5 passed it. Increasing RTP's goal bias made it faster, but its paths were still long. These conclusions are limited to this scene and sample size; small differences between medians may be due to chance.

Difficulty, hours and contributions

Exercise 1: difficulty 5/10, about 6 hours. Exercise 2: 6/10, about 7 hours. Exercise 3: 6/10, about 7 hours. The hardest parts were state ownership, choosing obstructed but solvable queries, and explaining why RRT sometimes failed despite finding short paths in other trials.

Darshon implemented and debugged RTP, managed memory, and analyzed performance and algorithm differences. Winson created the scenes and start–goal queries, generated Viser screenshots and saved paths, and configured and ran the benchmarks. Both tested RTP in both scenes, prepared the Part 1 write-up and package, and combined results, figures and explanations for the final report and submission.